# Summarize Class Activities

- Variable : A name that refers to a value but it can change.
  Example: student_name
- Constant: A value that stay the same during the program.
  Example: π = 3.14
- Function: A reusable block of code that perform one task, defined with def.
  Example: calculate_average ( )
- Module : A single file containing definitions (functions, classes, constants)
  Example: grade.py
- Package: A folder of module.
- Class : A blueprint for creating objects. It describes what an item will look like and what it can do.
  Example : my_class
- Method : A function defined inside a class.

## 2.+ Clean Code, PEP 8, and Debugging

1. Meaningful names: Give variables, functions and classes names that say what they hold or do. A ready should understand the code without guessing or reading comments.
   Example: total_price = quantity * unit_price
2. Small functions: Each functions should do one task.
   Example: read_scores(), calculate_average(), print_result()
3. DRY (Don't Repeat Yourself): Don't copy and paste the same code in several places. Put it in a function and call that function. When the logic changes, we need to update one place instead of hunting for every copy.
4. Comments (explain why): Use comments to explain the reason behind a decision, which the code can't show.
5. Docstrings: It is a description written in triple quotes """..."""" as the first line inside a module, class or function. It tells users what the code does, what inputs it takes and what it return.
6. Avoid magic numbers_replace them with named constant.

7. Debugging : Show the errors in code, so that helps save the time to fix it.

+ PEP 8 Rules
- Indentation: 4 spaces per level (no tabs)
- Line length: max 79 characters (72 for comments/docstrings)
- Blank lines: separate code into sections to give the code space.
- Imports: Put all imports at the top of the file, one per line, in three groups with a blank line between each group.
- Space in the right places:
  . put a space around operators
  . put a space after commas
  . put no space inside brackets
- Comparison: Writing cleaner, more readable logic when comparing variables in Python
  . check for None : Use is or is not instead of == or !=
- Consistent Quotes: Can use either single or double quotes.

+ Debugging : finding and fixing errors in code.
- Finds where things broke : When Python hits an error, debugging helps read the error message to find the exact line and file causing the problem.
- Fix different kinds of mistakes: It helps clean up grammar mistakes (syntax), stops the program from crashing while running (runtime), and fixes math or logical mistakes where the program runs but give wrong answer.
- Peeks inside the code: It freeze the program at any moment to see exactly what variables, lists or data look like right then.

- Save time : it lets we see inside running code instantly without making a mess to clean it up later.

Beside the lesson teacher told us to create our own reposity for this course to submit all tasks like TP, homework, assignment read slide and our project. After created it, drop the link to the group and called everyone to explain each point in the slide.